MANUAL TÉCNICO

Sistema de asignación de turnos

Arquitectura, base de datos, entornos, despliegue, cuentas, monitoreo y las tareas de mantenimiento más comunes, explicadas paso a paso para quien continúe con el sistema.

Código del documento	FTE-TUR-MT-02
Versión	1.2
Fecha	10 de noviembre de 2026
Elaboró	Pablo García · Desarrollador
Revisó / aprobó	Marlon Fuentes · Gestor de proyecto
Estado	Versión de entrega
Clasificación	Uso interno · Personal técnico

Historial de revisiones

Versión	Fecha	Cambios	Autor
1.0	06/11/2026	Primera versión, para la salida a producción.	Pablo García
1.1	10/11/2026	Rol Superadministrador, sección Cuentas de acceso, sesión de administrador de 1 hora y entrada con contraseña o código.	Pablo García
1.2	10/11/2026	Cambio de contraseña obligatorio en el primer inicio de sesión y restablecimiento de contraseña desde Cuentas de acceso.	Pablo García

Contenido

1. Introducción
2. Arquitectura
3. Tecnologías
4. Estructura del código
5. Base de datos
6. Entornos y variables
7. Instalación local
8. Flujo de cambios y despliegue
9. Migraciones
10. Cuentas y acceso
11. Monitoreo y registros
12. Respaldos y recuperación
13. Pruebas y calidad
14. Tareas de mantenimiento
15. Solución de problemas
16. Buenas prácticas de seguridad

1. Introducción

Este manual es para la persona que mantenga el sistema: corregir errores, hacer cambios, publicar nuevas versiones o atender un problema en producción. Se asume que conoce lo básico de JavaScript, Git y bases de datos. Para saber *qué* hace el sistema y sus reglas de negocio, lea primero la *Documentación del sistema* (FTE-TUR-DOC-01).

> **IMPORTANTE**
> El código del proyecto también está comentado pensando en el mantenimiento. Cuando este manual mencione un archivo, abrirlo y leer su comentario inicial suele ser el mejor siguiente paso.

Recursos

Recurso	Dónde
Aplicación en producción	`https://fuentelectric-asistence-table.vercel.app`
Repositorio	GitHub: `PabloGarcia2810/fuentelectric_asistence_table` (privado). Rama principal: `main`.
Hosting	Vercel (proyecto conectado al repositorio).
Base de datos y cuentas	Supabase (PostgreSQL + Authentication).
Correos de acceso	Resend.
Errores	Sentry, organización `fuentelectric`.

2. Arquitectura

Es una aplicación web **Next.js** desplegada en Vercel. No hay un backend separado: las páginas se generan en el servidor y los cambios (guardar, eliminar) se hacen con *Server Actions* que corren en el mismo servidor. Toda la información vive en PostgreSQL (Supabase) y se accede con Prisma.



Recorrido de una petición

1. **Proxy** (`src/proxy.ts` → `src/utils/supabase/middleware.ts`): renueva la sesión de Supabase y, si no hay sesión, manda al login.
2. **Página** (`src/app/tableros/<sección>/page.tsx`): comprueba rol y vigencia de la sesión (`lib/sesion.ts`), consulta la base y entrega los datos al componente de pantalla.
3. **Componente de pantalla** (`*Client.tsx`): formularios, filtros y diálogos en el navegador.
4. **Acción** (`actions.ts`): al guardar, vuelve a comprobar que sea admin (`lib/auth.ts`), valida con las reglas de `lib/reglas-turno.ts` y escribe en la base dentro de una transacción. Devuelve `null` si todo salió bien o el texto del error.

3. Tecnologías

Pieza	Versión	Para qué se usa
Next.js (App Router)	16.2	Páginas, acciones del servidor y rutas /api. En esta versión el “middleware” se llama `proxy`.
React	19.2	Interfaz.
TypeScript	5	Todo el código de `src/`.
Prisma + adaptador pg	7.8	Acceso a la base y migraciones.
Supabase	—	PostgreSQL y autenticación (contraseña y código por correo).
Tailwind CSS + shadcn/ui (Radix)	4	Estilos y componentes (diálogos, menús, selects).
ExcelJS · jsPDF	4.4 · 4.2	Plantilla y carga masiva de Excel; reportes en Excel y PDF.
Resend	6	Envío del código de acceso por correo.
Sentry	10	Registro de errores en navegador y servidor.
Node.js · pnpm	24 · 12	Entorno de ejecución y gestor de paquetes.

4. Estructura del código

```
src/
├── proxy.ts                    Se ejecuta antes de cada página (sesión y redirecciones)
├── app/
│   ├── page.tsx                Portada pública
│   ├── login/                  Inicio de sesión (contraseña / código por correo)
│   ├── not-found.tsx           Página 404
│   ├── tableros/               Panel (requiere sesión)
│   │   ├── page.tsx            Dashboard
│   │   ├── asistencias/        Asignación de turnos, calendario, carga masiva
│   │   ├── usuarios/           Trabajadores
│   │   ├── areas/  vehiculos/  Catálogos
│   │   └── historial/          Auditoría y reportes
│   └── api/
│       ├── asistencias/        Plantilla de Excel y carga masiva
│       └── historial/          Descarga de reportes PDF y Excel
├── lib/
│   ├── reglas-turno.ts         Reglas de negocio de los turnos (un solo lugar)
│   ├── puestos-no-asignables.ts  Puestos que nunca reciben turnos
│   ├── horarios-turno.ts       Horarios, etiquetas y compatibilidad
│   ├── estados-vehiculo.ts     Estados de vehículo y sus colores
│   ├── sesion.ts · auth.ts     Sesión y permisos
│   ├── form-validation.ts      Lectura de formularios y fechas (hora de Guatemala)
│   ├── carga-masiva.ts         Lectura y validación del Excel
│   ├── reportes-exportacion.ts Datos de los reportes
│   └── reportes/{pdf,excel}/   Dibujo de cada sección del reporte
└── components/                 Componentes compartidos y del panel
prisma/
├── schema.prisma               Estructura de la base (comentada)
├── migrations/                 Historial de cambios de la base
└── seed.mjs                    Datos de prueba (solo staging/desarrollo)
scripts/preparar-entrega.mjs    Respaldo/limpieza usados en la entrega
tests/                          Pruebas automáticas (sin base de datos)
docs/manuales/                  Esta documentación
```

CONSEJO

Cada sección del panel sigue el mismo patrón: `page.tsx` (datos, en el servidor) · `*Client.tsx` (pantalla) · `actions.ts` (cambios). Si sabe dónde está una, sabe dónde están todas.

5. Base de datos

Tabla	Contenido y notas
`Usuario`	Cuentas que pueden entrar. El `id` es el mismo de Supabase Auth. `rol`: `SUPERADMIN`, `ADMIN` o `TRABAJADOR`. `debeCambiarContrasena`: si está en `true`, debe elegir una contraseña nueva al entrar.
`Trabajador`	Personal que recibe turnos: nombre, puesto (texto libre), estado, horario establecido y unidad fija opcional.
`Area`	Zonas operativas. Nombre único y estado.
`Vehiculo`	Unidades: código único, tipo (`CARRO` / `CAMION`), capacidad (`noPersonas`) y estado.
`TurnoDiario`	Un turno. Guarda la relación con trabajador, área y vehículo y además una copia de sus nombres. Único por trabajador + fecha + jornada.
`SnapshotTurno`	Auditoría en texto, sin relaciones: sobrevive aunque el turno se borre.

IMPORTANTE

Fechas de turno: la columna `fecha` se guarda a las **12:00 UTC** del día (06:00 en Guatemala), así el día nunca cambia al convertir zonas horarias. Para buscar turnos de un día use `rangoFechaTurno()` de `lib/form-validation.ts`; no arme las fechas a mano.

Conexiones: la aplicación usa `DATABASE_URL` con el *pooler* de Supabase en modo *transaction* (puerto 6543) y un máximo de 5 conexiones por instancia (`DATABASE_POOL_MAX`). Las migraciones usan `DIRECT_URL` en modo *session* (puerto 5432).

Seguridad por filas (RLS): está activada en todas las tablas sin políticas, de modo que la API pública de Supabase no puede leer nada. La aplicación usa el rol `postgres`, que no se ve afectado.

6. Entornos y variables

Las variables se configuran en Vercel (*Settings → Environment Variables*) y, para trabajar en local, en un archivo `.env` (plantilla: `.env.example`). **Nunca** se suben al repositorio.

Variable	Para qué sirve
`NEXT_PUBLIC_ENTORNO`	`produccion`, `staging` o `desarrollo`. Fuera de producción muestra una etiqueta en pantalla y separa los errores en Sentry.
`DATABASE_URL`	Conexión de la app (pooler, puerto 6543).
`DIRECT_URL`	Conexión para migraciones (puerto 5432).
`NEXT_PUBLIC_SUPABASE_URL` · `NEXT_PUBLIC_SUPABASE_ANON_KEY`	Proyecto de Supabase (públicas).
`SUPABASE_SERVICE_ROLE_KEY`	Clave secreta que genera los códigos de acceso. Solo en el servidor.
`RESEND_API_KEY`	Envío de correos con el código.
`SENTRY_AUTH_TOKEN`	Opcional: sube los *source maps* a Sentry al compilar.
`DATABASE_POOL_MAX`	Opcional: conexiones por instancia (5 por defecto).

PRECAUCIÓN

Hoy el `.env` local del equipo de desarrollo apunta a la base de **producción**. Cualquier comando de base de datos que se ejecute en local (`pnpm db:migrate`, `pnpm entrega …`, Prisma Studio) cambia datos reales. Lo recomendado es crear un proyecto de Supabase de staging y usar ese en el `.env`.

7. Instalación local

1. Instale Node.js 24 y pnpm 12, y clone el repositorio.
2. Instale las dependencias (también genera el cliente de Prisma):

```
pnpm install
```

3. Copie `.env.example` a `.env` y complete los valores (de preferencia, de un proyecto de staging).
4. Si la base está vacía, cree las tablas y, solo en staging, cargue datos de prueba:

```
pnpm db:migrate
pnpm db:seed
```

5. Inicie el servidor de desarrollo y abra `http://localhost:3000`:

```
pnpm dev
```

8. Flujo de cambios y despliegue

Vercel publica automáticamente lo que llega a `main`. Por eso todo cambio pasa por una rama y un *pull request*:

1. Cámbiese a `main` y actualícela (*Fetch* y *Pull* en GitHub Desktop).
2. Cree una rama nueva **desde main** con un nombre descriptivo: `fix/…` para correcciones, `feat/…` para funciones nuevas, `docs/…` para documentación.
3. Haga el cambio y verifique en local: `pnpm lint`, `pnpm typecheck`, `pnpm test` y `pnpm build`.
4. Publique la rama y abra el *pull request* hacia `main`. GitHub Actions corre las mismas comprobaciones.
5. Si el cambio necesita una migración, aplíquela **antes** de unir (sección 9).

6 Una vez unido el PR, Vercel publica en uno o dos minutos. Revise la página en producción.

> **IMPORTANTE**
> No use la rama `dev`: quedó atrasada respecto a `main` y los cambios hechos ahí generan conflictos. Si un PR muestra conflictos, lo más seguro es rehacer el cambio en una rama nueva desde `main`.

Volver a una versión anterior: en Vercel, *Deployments* → elija la última versión que funcionaba → *Promote to Production* (o *Instant Rollback*). Esto revierte el código, no la base de datos.

9. Migraciones de base de datos

Cada cambio de estructura de la base es una migración en `prisma/migrations/`. La última es `20261110120000_rol_superadmin`. Revise con `pnpm db:status` cuáles faltan en producción.

1. Modifique `prisma/schema.prisma`.
2. Genere la migración contra una base de pruebas (nunca contra producción):

```
pnpm prisma migrate dev --create-only --name descripcion_corta
```

3. Revise el archivo `migration.sql` generado. Si quita valores o columnas, primero debe mover los datos existentes (vea `20261106180000_simplificar_estados` como ejemplo).
4. Revise qué falta aplicar en producción y aplíquelo **antes** de publicar el código que lo usa:

```
pnpm db:status
pnpm db:migrate
```

5. Una a `main` el PR con el código.

> **NOTA**
> ¿Por qué antes? El código anterior suele funcionar con la base ya migrada, pero el código nuevo puede fallar si la base todavía no tiene el cambio.

10. Cuentas y acceso

Una cuenta vive en **dos lugares**: el usuario en Supabase Auth (correo, contraseña y sesión) y su fila en la tabla `Usuario` (nombre y rol) con el mismo id. Las acciones de `src/app/tableros/cuentas/actions.ts` cambian los dos a la vez y, si el segundo paso falla, deshacen el primero.

Rol	Permisos y forma de entrar
`SUPERADMIN`	Todo lo de `ADMIN` y además la sección **Cuentas de acceso**. Contraseña o código; sesión de 1 hora.
`ADMIN`	Turnos, catálogos e historial. Contraseña o código; sesión de 1 hora.
`TRABAJADOR`	Solo el Dashboard. Solo código; sesión de 30 días.

Las reglas de permisos están en `src/lib/roles.ts` (`esAdministrador`, `esSuperadmin`). No compare el texto del rol a mano en el resto del código.

Crear, editar o eliminar una cuenta

Se hace desde el panel, con una cuenta de superadministrador: *Cuentas de acceso*. Ya no hace falta entrar a Supabase. El sistema aplica estas protecciones:

- Nadie puede eliminarse ni cambiarse el rol a sí mismo.
- Siempre debe quedar al menos un superadministrador.
- Las cuentas de administrador necesitan contraseña (mínimo 8 caracteres); además pueden entrar con código.
- Si un correo ya existía en Supabase Auth sin fila en `Usuario` (una cuenta a medias), al crearla se reutiliza.

Cambio de contraseña obligatorio

La columna `Usuario.debeCambiarContrasena` la activa el superadministrador con la casilla *Pedir cambio de contraseña al iniciar sesión* (al crear la cuenta o al editarla). Mientras esté en `true`:

- Al iniciar sesión (con contraseña o con código) la persona va a `/cambiar-contrasena` en lugar de `/tableros`.
- `requerirSesion` (`src/lib/sesion.ts`) la regresa ahí desde cualquier página del panel, y `verificarAdmin` / `verificarAdminApi` (`src/lib/auth.ts`) rechazan sus acciones y descargas.
- La contraseña nueva se guarda con la sesión de la propia persona (`supabase.auth.updateUser`), así Supabase aplica sus reglas y la sesión sigue abierta. Después la columna vuelve a `false`.

Solo aplica a administradores y superadministradores; en una cuenta de trabajador la casilla no aparece y el valor se guarda en `false`.

Si no queda ningún superadministrador

Solo pasaría si alguien cambia la base a mano. Se recupera desde Supabase → *SQL Editor*:

```
update "Usuario" set rol = 'SUPERADMIN' where email = 'correo@fuentelectric.com';
```

Otras operaciones

- **Restablecer la contraseña de un admin:** en *Cuentas de acceso* → editar → marcar *Pedir cambio de contraseña al iniciar sesión*. La persona entra con código por correo y elige la nueva. Si se prefiere, también se le puede escribir una contraseña temporal en *Nueva contraseña*.
- **Quitar un cambio pendiente sin que lo haga:** editar la cuenta y desmarcar la casilla, o en el *SQL Editor*: `update "Usuario" set "debeCambiarContrasena" = false where email = 'correo@fuentelectric.com';`
- **Quitar el acceso:** eliminar la cuenta en *Cuentas de acceso*. Se borra de Supabase Auth y de `Usuario`, y se cierran sus sesiones. Los turnos y la auditoría no se ven afectados.

Valores de seguridad que se pueden ajustar

Valor	Dónde
Duración de la sesión (1 hora administradores / 30 días trabajador)	`src/lib/auth.ts`
Intentos de inicio de sesión y códigos por correo	Constantes al inicio de `src/app/login/actions.ts`
Diseño y texto del correo con el código	`enviarCorreoOtp` en `src/app/login/actions.ts`

> **NOTA**
> El límite de intentos se guarda en la memoria de cada instancia del servidor. Frena el abuso casual, pero no es un límite global; si se necesitara, habría que usar un almacén compartido (por ejemplo, una tabla o Redis).

11. Monitoreo y registros

- **Sentry** recibe los errores del navegador, del servidor y del proxy, separados por entorno. Configuración: `src/instrumentation-client.ts`, `sentry.server.config.ts` y `sentry.edge.config.ts`. Recomendado: crear una alerta por correo para errores nuevos en *Alerts* → *Create Alert*.
- **Vercel** → **Logs** muestra los mensajes del servidor (por ejemplo, si falló el envío de un correo).
- **Supabase** → **Reports** muestra el uso de la base y las conexiones.
- **Resend** → **Emails** muestra si cada correo de código se entregó o rebotó.

12. Respaldos y recuperación

- Revise en Supabase (*Database* → *Backups*) qué respaldos automáticos incluye el plan contratado y por cuántos días se guardan.
- Para un respaldo propio de toda la base, use `pg_dump` con la conexión de `DIRECT_URL` y guarde el archivo en un lugar seguro de la empresa:

```
pg_dump "$DIRECT_URL" --no-owner --format=custom --file=respaldo-AAAA-MM-DD.dump
```

- Los reportes de *Historial* (Reporte Completo en Excel) sirven como respaldo legible de la programación de un periodo.
- `pnpm entrega respaldar` copia los catálogos (áreas, vehículos, trabajadores y usuarios) a la carpeta `respaldos/`, que no se sube a Git porque contiene datos personales.

> **PRECAUCIÓN**
>
> **No ejecute `pnpm entrega limpiar`.** Borra todas las asistencias y el historial; solo se usó para dejar la base limpia antes de la entrega.

13. Pruebas y calidad

Comando	Qué comprueba
`pnpm lint`	Reglas de estilo y errores comunes (ESLint).
`pnpm typecheck`	Tipos de TypeScript, incluida la coincidencia entre los estados del código y los de la base.
`pnpm test`	23 pruebas automáticas: reglas de turno, permisos, carga masiva, reportes, fechas y sesión. No usan la base real.
`pnpm build`	Que la aplicación compile como en producción.

GitHub Actions (`.github/workflows/ci.yml`) corre los cuatro en cada *push* a `main` y en cada *pull request*. No una un PR con el CI en rojo.

14. Tareas de mantenimiento

Necesito...	Qué hacer
Que otro puesto no reciba turnos	Agréguelo a la lista de `src/lib/puestos-no-asignables.ts`. No necesita migración.
Un horario nuevo (ej. 05:00–17:00)	Agregue el valor a `enum HorarioBase` en el esquema y cree la migración; luego agréguelo en `src/lib/horarios-turno.ts` (etiqueta y rango) y en el mapa de `src/lib/carga-masiva.ts`.
Agregar o quitar un estado de vehículo	Cámbielo en `enum EstadoVehiculo` (con migración) y en `src/lib/estados-vehiculo.ts`. TypeScript avisa si no coinciden.
Cambiar una regla de turno	Solo en `src/lib/reglas-turno.ts`: la usan el formulario, la edición y la carga masiva. Agregue o ajuste su prueba en `tests/`.
Cambiar textos de una pantalla	En el `*Client.tsx` de la sección.
Cambiar colores del panel	Variables al inicio de `src/app/tableros/dashboard.css`.
Cambiar el diseño de un reporte	Datos: `src/lib/reportes-exportacion.ts`. Dibujo: `src/lib/reportes/pdf/` o `src/lib/reportes/excel/`.
Actualizar dependencias	En una rama: `pnpm update`, luego las cuatro comprobaciones y una prueba manual. Lea las notas de versión de Next.js antes de un cambio mayor.

15. Solución de problemas

Síntoma	Causa probable	Solución
Un trabajador no recibe el código	Correo mal escrito, en spam, o falla de Resend.	Revise la carpeta de spam y Resend → *Emails*. Compruebe que el correo esté en `Usuario` en minúsculas y que exista en Supabase Auth.
"No se encontró ninguna cuenta asociada..."	Falta la fila en `Usuario`.	Créela (sección 10).
El admin vuelve al login seguido	La sesión de administrador dura 1 hora.	Es lo esperado. Se ajusta en `src/lib/auth.ts`.
Una persona no aparece al asignar turnos	No está *Activo*, o su puesto está en la lista de no asignables.	Revise su estado y cómo está escrito su puesto en Trabajadores.
`typecheck` marca un error en `estados-vehiculo.ts` tras cambiar de rama	El cliente de Prisma quedó generado con otra versión del esquema.	`pnpm prisma generate` y repetir.
Error en producción después de publicar	Falta aplicar una migración, o un error de código.	`pnpm db:status`; si falta, `pnpm db:migrate`. Si no, revise Sentry y, si es urgente, vuelva a la versión anterior en Vercel.
"Too many connections" o lentitud	Demasiadas conexiones a la base.	Confirme que `DATABASE_URL` usa el puerto 6543 y revise `DATABASE_POOL_MAX`.
La carga masiva rechaza filas	Nombres que no coinciden o reglas de turno.	El resultado indica fila y motivo. Use la plantilla recién descargada y copie los nombres de su hoja de catálogos.
Conflictos al abrir un PR	La rama se creó desde una rama atrasada (como `dev`).	Rehaga el cambio en una rama nueva desde `main` actualizada.

16. Buenas prácticas de seguridad

- Las claves (`SUPABASE_SERVICE_ROLE_KEY`, `RESEND_API_KEY`, conexiones) solo van en Vercel y en el `.env` local; nunca en el código, en capturas ni en chats.

- Toda acción nueva que cambie datos debe empezar con `verificarAdmin()` (o `verificarSuperadmin()` si es de cuentas), y toda página de administración con `requerirAdminPagina()`.
- Mantenga al menos **dos administradores** activos para no quedarse sin acceso.
- Cuando alguien deje la empresa, quite su acceso el mismo día (sección 10).
- La carpeta `respaldos/` tiene datos personales: no la comparta ni la suba a Git.
